BPM and linearity characterization
Usage and mathematics, version 1

Wenu-imaging

10 November 2026

Executive summary

For each detector, ingest calibrations and raw planes once, scan-correct, subtract the supplied master from flats, trim, and retain one processed cube. Create a robust-fit BPM if no input BPM is supplied; then measure linearity ROIs and direct sites from the same cube. Release it before the next detector. Linearity is measurement only; no response correction or gain conversion is applied.

Terminal usage

Run from orchestration. The launcher admits metadata by default; add `--execute` to process. It generates frozen native bindings and sets niceness to at least 10. Project TOML supplies raw, calibration, reduced, work and log roots. Use a fresh empty directory for the recipe's `run`.

```
python /path/to/tools/run-bpm-linearity.py --config RECIPE.toml
python /path/to/tools/run-bpm-linearity.py --config RECIPE.toml \
    --execute --qc false
```

The underlying native command takes already generated bindings:

```
nice -n 10 mefpp_bpm_linearity_recipe cal bpm-linearity \
    --config RECIPE.toml --bindings NATIVE.toml [--qc true|false]
mefpp_bpm_linearity_recipe --help
```

CLI option	Meaning / default
`--config FILE`	Required recipe TOML (launcher and native command).
`--execute`	Launcher only; absent means metadata admission, no reduction.
`--bindings FILE`	Native only; required frozen bindings from the launcher.
`--qc true|false`	Optional override; CLI > recipe `qc` > true.
`--help`	Usage, no processing.

Recipe settings

Templates: `config/recipes/omegacam-bpm-linearity.example.toml`

`config/recipes/wfi-bpm-linearity.example.toml`. Replace dependency paths, qualified executable and revision. These are authoring templates: WFI still requires a reviewed paired-flat archive and master handoff.

Field	Default / requirement
`project`, `run`	Required; project path and fresh run below reduced.
`qc`	true; false skips optional full-frame BPM QC and MAD.
`native_recipe`	Required executable and exact compiled source revision.
`assembly.reference_detector`	Required selected detector; among created BPMs if any.
`preparation.detector_toml`	Required detector identities/geometry.
`preparation.processing_toml`	Required scan/trim/ROIs and processing controls.
`preparation.input_bpm`	`archived`; `omitted` enables new BPM creation.
`bpm.acquisition`	Required: `quick` or `provisional`.
`bpm.output_mode`	Required: `new` or `combined`.
`bpm.response_low/high`	0.5 / 1.5; finite, $0 <$ low $<$ high.
`bpm.upper_fraction`	Required, $(0, 1]$; example 0.85.
`bpm.minimum_span_fraction`	Required, $(0, 1]$; example 0.15.
`bpm.maximum_pair_difference`	Required, ≥ 0; example 0.02.
`detectors`	Required ordered list: ID, sample ASDF, handoff, reference ROI IDs.
`calibration_bindings`	Empty; optional byte-identical source/path/SHA256 copies.

`with_mad` is an explicit processing-TOML control; QC true does not force it on. QC false retains requested ROI means, variances, populations, direct samples, traces and provenance. Structural checks remain enabled.

Processing and populations

A supplied master must match the archived scan/trim/stack configuration and reside under the calibration root. A supplied BPM is loaded once and skips creation, explicitly logged. With `input_bpm="omitted"`, its historical lineage remains intact but it is absent from current inputs. Previously admitted direct samples must remain exact; newly admitted samples are reported separately. ROI populations may increase. A generated candidate is never fed back into this run's measurement mask.

For frame f and raw row y, sort enabled scan values $v_{f,y,(k)}$ and reject l low and h high reads:

$$O_f(y) = \mathrm{pix}\left[\frac{\sum_{k=l+1}^{n-h} v_{f,y,(k)}}{n-l-h}\right], \tag{1}$$

$$U_f(x,y) = \mathrm{pix}[D_f(x,y) - O_f(y)], \tag{2}$$

$$S_f(p) = \mathrm{pix}[U_f(p) - B(p)] \quad \text{(flats only)}. \tag{3}$$

Scan summation/division use double precision; pix converts to native `pixelvalue`. Preserve both subtraction roundings. There is no new scan fit or smoothing. Trim coordinates are FITS 1-based inclusive; active row j maps to raw row $j + y_{\min} - 1$. The master receives no further scan correction. Bias-role frames have no master subtraction. Active BPMs do not mask electronic scan reads.

The common ROI population $\mathcal{P}$ passes the declared input mask, finite-value and raw-saturation checks in both flats and both reference biases. The saturation limit is an explicit processing control (OCAM: 65534 ADU). For $N = |\mathcal{P}|$:

$$\overline{S}_i = N^{-1} \sum_{p\in\mathcal{P}} S_{ip}, \qquad s_i^2 = \frac{\sum_{p\in\mathcal{P}} (S_{ip} - \overline{S}_i)^2}{N-1}, \tag{4}$$

$$d_p = \mathrm{pix}[S_{1p} - S_{2p}], \qquad s_d^2 = \frac{\sum_{p\in\mathcal{P}} (d_p - \overline{d})^2}{N-1}. \tag{5}$$

Scatter is unavailable for $N < 2$. Corrected counts must equal admitted raw counts. Transports retain member variances, difference variance and $s_d^2/2$; spatial scatter is not a noise model or a fitting weight. Units remain ADU and ADU^2; covariance and master variance are unavailable.

Candidate BPM and response limits

The existing robust fitter in `mefpp2/src/iproc/detector_response.c` classifies normalized pixel response R using `MEF.BPM.LGAINTHR` and `MEF.BPM.HGAINTHR`. Recipe `response_low/high` override these for the current run only; the input master's original snapshot is preserved. $R \leq$ low sets reason 4, $R \geq$ high sets reason 8: unflagged response lies strictly inside the limits. The existing native configuration stores limits in float32. These ratios are *dimensionless response*, not gain in electrons/ADU. Other reason bits and robust-fit arithmetic are unchanged.

Reference ROIs, acquisition mode, usable signal range, pair mismatch and coverage are saved with suitability assessments. Failed suitability does not abort the subsequent measurements or establish scientific acceptance. A zero reason byte means no detected reason, not certification.

Native recipe and source map

The entry point `mefpp2/ins/mef/recipes/mefpp_bpm_linearity.c` follows `mefpp_bias.c`: initialize/start; prepare/admit/begin; detector loop; verify/publish/complete; common failure cleanup. Source landmarks:

1. `BPM-LINEARITY-ADMISSION`: `mef_bpm_linearity_config.c`, `mef_bpm_linearity_inputs.c`, `mef_bpm_linearity_bindings.c`. Validate project roots, ordered bindings, frozen configurations and dependencies.
2. `BPM-LINEARITY-DETECTOR-LOOP`: `mef_bpm_linearity_process.c`. `BPM-LINEARITY-INGESTION`: `mef_detector_cube.c` owns loaded calibrations, raw/active planes and row traces; services borrow them.
3. `BPM-LINEARITY-CANDIDATE`: `mef_bpm_command_from_cube`; `mef_cal_bpm.c` and `detector_response.c` own fitting.
4. `BPM-LINEARITY-MEASUREMENTS`: `mef_linearity_prepare_from_cube`, `mef_linearity_pairs.c` and `mef_linearity_prepare.c`; row arithmetic uses `mef_overscan_correction_traced` and difference statistics use `image_pair_stats_checked`.
5. `BPM-LINEARITY-PUBLICATION` and `BPM-LINEARITY-PRESERVATION`: `mef_bpm_linearity_products.c`, `mef_bpm_linearity_hash.c`, shared native ASDF writer and MEF assembly.

C helper files above reside under `mefpp2/ins/mef/common`, except the image statistics/fitting kernels under `mefpp2/src/iproc`. Pycavi's `pycavi_bpm_linearity_native_recipe.py` admits metadata, invokes one native command and independently reads saved products. It contains no detector processing loop. No Python pixel reduction or ASDF writer substitutes for the native producer.

Products and verification

Each detector retains preparation ASDF with input/HDU identities, native measurements, traces, effective configuration, original dependency graph and availability flags. Newly created BPMs have FITS, ASDF and logs. Final MEF/ASDF assembly includes only newly created BPMs; an all-supplied-mask run has no new BPM MEF. The manifest reports completion for inspection, not scientific approval.

Raw observations, calibrations and historical ASDFs are unchanged. Image planes are loaded/corrected once numerically; whole-file preservation SHA256 passes still read inputs before and after the batch. Saved readers require neither raw pixels nor native execution. A zero native exit, completion receipt and independent saved-product checks are required before acceptance.

Qualification uses tiny analytical scan/ROI fixtures, borrowed-cube ownership checks, exact saved-product comparison and independent native-ASDF readers. The supplied-mask CCD65 path passed Mac comparison. New recipe controls and explicit archived-mask omission require their Mac gate before engine selection. The stopped 32-detector run is preserved. Physical linearity correction, cross-talk and uncertainty modeling remain deferred. Scientific design:

`docs/developer/design/linearity-preparation-overscan-asdf-v1.md`.

Coordinator design: `docs/developer/design/common-recipe-coordinator-v1.md`.